

Accounts and Security

Making and changing accounts, passwords, and who may enter an account.

These are the verbs that decide **who may use this installation and what they may do with it**. They are the administrator's set, they are the smallest set on the machine, and almost all of them need more than the verb before they will do anything.

This document is separate so that it can be withheld. Everything in the administrator set describes verbs an ordinary account does not have and cannot run. It is a complete set on its own and **links to nothing outside itself**, so that handing somebody the user documentation without this never leaves them at a page that is not there. Where a user-set page is worth naming, it is named in words rather than linked.

SD folds case, so a command may be typed in either case. Commands are shown here in lower case. In the tables, *italics* mark something you supply and **bold** marks a word typed as it stands; braces mark an optional part.

Read this before anything else: there are two gates, not one

Having the verb is not having the right to use it.

the tier	decides whether the account's VOC has the verb at all. Only an <code>administrator</code> account has these names; in any other account they are not recognised
elevation	decides whether the verb does anything. Almost every verb here begins by testing it, and stops if the session is not elevated

In an administrator account, from an ordinary unelevated session:

```
:list.grants don
Command requires administrator privileges
:delete.account sdsys
Command requires administrator privileges
:modify.password sdsys
Command requires administrator privileges
```

Every one of those refusals came before the command was looked at. `delete.account sdsys` would have been refused anyway — you cannot delete `SDSYS` — and it never got that far.

What counts as elevated is a Windows question and is fixed when SD starts. The token a process gets is decided at process creation, so there is nothing you can type inside a running session to elevate it. **Start SD from an elevated terminal**, or accept the UAC prompt that `logto sdsys` raises. The three verbs on this page that do *not* test elevation are `clean.account`, `config` and `update.accounts`, and each of them acts only on the account you are already standing in or on your own session.

Making an account: `create.account`

```
create.account user <name> {administrator | programmer}
                        <ssh | api | both | none> {no.query}

create.account group <name> {no.query}

create.account other <name> <pathname> {no.query}
```

<code>user</code>	an SD account and a Windows local account to sign in as
-------------------	--

<code>group</code>	a shared workspace with no Windows account, reached only with <code>logto</code>
--------------------	--

<code>other</code>	an SD account over a directory you name
--------------------	---

One of `ssh`, `api`, `both`, `none` is required for a user account and there is no default. An account meant to be reached only with `logto` says `none` and means it. An `administrator` account is given both routes and takes no keyword. Omitting the tier gives a **standard** account.

It prompts for a password and `no.query` does not suppress that. `no.query` covers the confirmation, not the credential — a password is never an argument anywhere in SD. `create.account user` **therefore cannot be driven from a script**, and a group account, which has no password, can.

`sh-on` and `os-on` may be added to give the new account the `sh` verb and `OS.EXECUTE`; see `modify.account` below, which has both and their `-off` forms.

THE ACCESS MODEL IN FOUR SENTENCES

Administrators have API access and `OS.EXECUTE` access automatically. Programmers and standard users do not, unless it is given specifically. **API access does not give `OS.EXECUTE` access. And an administrator's ssh and API access stops at this machine.**

The fourth sentence is not a grant and there is no keyword for it. An administrator keeps both routes permanently — they cannot be taken away — and neither reaches past this computer: a sign-in from anywhere else is refused after the password has been checked.

An administrator may not sign in to this machine from another one.

Administration happens at the console, or through a remote desktop or remote-control product installed as a service, because those are the sessions Windows can show a consent prompt on. **Only the administrator tier is affected;** programmers and standard users reach the machine remotely on whatever routes they were given.

The third sentence is the one a reader will otherwise get wrong, because the two grants look like one grant. They are not:

`api` says who may **connect**

`os-on` says who may reach the **operating system**

Holding the first has never implied the second. They are independent settings and are checked in different places.

Neither can be taken away from an administrator. `modify.account` refuses `os-off` and `sh-off` for an administrator account, exactly as it refuses `ssh`, `api` and `none`.

WHAT OS-ON ACTUALLY COSTS

Granting `os-on` to a non-administrator is a larger decision than it looks.

That account reaches the operating system over the API as well as locally — and over the API the session's process token is LocalSystem. On a programmer-tier account over a remote API connection, the operating system reports the session as `nt authority\system`.

This is accepted behaviour rather than a defect, and it follows from the way SD's listener creates a session. It is stated here because it is not something a reader should have to discover: `os-on` on a non-administrator account with API access gives that account the operating system, as LocalSystem, from any machine that can reach the API port.

The full account of what creating a user account does to Windows — the groups, the disabled-then-enabled login, the console denial — is in the SD Core for Windows release documentation, under *Account types*, and is not repeated here.

Changing one: `modify.account`

```
modify.account account standard | programmer | administrator | suspended
modify.account account standard | programmer  ssh | api | both | none
modify.account account ssh | api | both | none
modify.account account sh-on | sh-off
modify.account account os-on | os-off
modify.account account add | delete user.name
```

THE TIER MOVES IN ANY DIRECTION

`standard`, `programmer`, `administrator` and `suspended` are four names for one state, and any of them can be reached from any other with no intermediate step. The account's VOC is rewritten there and then, not at its next login.

`suspended` denies access and changes nothing else. The VOC is left as it is, no Windows group membership moves, and the tier it displaced is remembered so that the way back knows where the account is coming from. **There is no `resume` keyword** — coming back names the destination tier, and naming one on a suspended account lifts the suspension into it.

Three doors refuse a suspended account: signing in, `logto`, and the API.

Leaving `administrator` takes three things away and one must be named. Windows `Administrators` membership and the operating-system access record go by themselves, because the account held them *because* it was an administrator. **ssh and the API were a rule and now have to be a choice**, so the second form above is compulsory: `modify.account don programmer both`.

THE OTHER KEYWORDS

<code>ssh</code> <code>api</code> <code>both</code> <code>none</code>	what the account's remote access is . Absolute: <code>api</code> takes ssh away
---	--

<code>sh-on</code> <code>sh-off</code>	the <code>sh</code> verb and <code>!</code>
--	---

<code>os-on</code> <code>os-off</code>	<code>OS.EXECUTE</code> and the screen editors
--	--

<code>add</code> <code>delete</code> <i>user</i>	put a Windows user in, or out of, the account's group
--	---

The first group is absolute and the second is not. `ssh`/`api`/`both`/`none` are four names for one state, so whatever is not named is withdrawn. `sh-on` and `os-on` are two independent switches over two separate settings, so `sh-off` says nothing about `OS.EXECUTE` and leaves it alone.

All of these refuse an administrator account. An administrator has full access and there is no way to turn any of it off — the way to reduce one is to move it to another tier, which is what the tier keywords are for. A group account is refused too: it has no Windows user to put in a group.

Passwords: `modify.password`

```
modify.password {account}
```

With no account name it changes **your own**, and asks for the current password first. With one it changes somebody else's, which needs elevation and does not ask for the old password — an administrator resetting a forgotten password does not know it.

The password is never an argument, and a trailing token is refused rather than ignored:

```
:modify.password don hunter2
```

A password is never given on the command line; MODIFY.PASSWORD prompts for it

That refusal is the point of the verb's design. The older behaviour set the password from the prompt and threw the extra word away without a word, so every visible sign said it had worked — while the password had already reached SD's command stack, and a shell's history and process list if the verb was reached from one. Refusing does not put it back, but it says so.

The prompts are hidden, asked twice, and the account must already exist in the register. `modify.password` **cannot be scripted**, by design.

Who may enter an account: `grant`, `revoke`, `list.grants`

```
grant account to user
revoke account from user
list.grants account
grant account          the same as list.grants
```

The grant is Windows group membership and nothing is written to the account record. Every SD account has a Windows group — `sdu_name` for a user account, `sdg_name` for a group account — and entry to the account **is** membership of that group. These three verbs edit that group and read it back.

<i>There is no Windows account named %1</i>	<code>grant</code> to somebody who does not exist
---	---

<i>%1 is already a member of group %2</i>	<code>grant</code> to somebody who already has it
---	---

<i>%1 has not been granted account %2</i>	<code>revoke</code> from somebody who has not
---	---

<i>Account not registered in ACCOUNTS file</i>	the account name is not one of SD's
--	-------------------------------------

<i>Account %1 has no Windows group recorded</i>	a record predating the group model; it is refused, not guessed at
---	--

The account's own user is listed like any other member. It is not a grant — creating the account put them there — but hiding it would make the listing disagree with the Windows group it is reporting.

A successful `grant` or `revoke` writes an audit record to SD's audit trail, stamped with the user and process that did it. The identity is stamped by SD and is not passed in, so a caller cannot get it wrong or forge it. Windows records the group change in its own security log as well, and the two are independent records of the same act.

`modify.account` ***account*** `add / delete` ***user*** makes the same group change and writes no audit record.

Prefer `grant` and `revoke` when there is a choice: they say what they mean and they leave a trail.

Continued in

Account Maintenance — `clean.account`, `update.accounts`, `[locked]`, `config`, `set.date` and `delete.account`.